

Animate JS

Éditeur d'animation vectorielle image par image

Documentation complète — scripts, CreateJS, exports et intégration jeu

Version 3 — Octobre 2026 · Licence MIT · TweenJS_simpleV.0.2

Sommaire

1. Bien démarrer
2. Tour de l'interface
3. Les scripts : le contrat
4. Exploiter CreateJS dans les scripts
5. Événements et interactions
6. Recettes prêtes à tester
7. Exporter : JSON, HTML, compilé
8. Utiliser un MovieClip dans votre propre jeu
9. Dépannage
10. Annexes : raccourcis et limites connues

1. Bien démarrer

Animate JS (projet TweenJS) est un éditeur d'animation vectorielle image par image, inspiré d'Adobe Animate/Flash : calques, images clés, interpolation de mouvement, morphing de courbes, symboles, ossature avec IK — le tout en JavaScript vanilla (Konva.js pour le rendu Canvas, Vite pour le build), sans framework.

1.1 Lancer l'application

```
npm install      « une seule fois »
npm run dev      « ouvre http://localhost:5173 »
```

Deux formes de l'application existent, mises à jour ensemble à chaque modification :

Version	Où	Usage
Principale	<code>src/</code> , servie par Vite	Développement, build <code>dist/</code>
PureVanilla	<code>Animate_JS_PureVanilla/</code> (HTML + bundle statiques)	Déploiement statique, aucune dépendance CDN (Konva et CreateJS sont des copies locales dans <code>libs/</code>)

1.2 Créer sa première animation en 60 secondes

1. Sélectionne l'outil **Rectangle** (**R**) et dessine un carré sur la scène.
2. Dans la timeline, clique une cellule plus loin (ex. image 12) et appuie sur **F6** : nouvelle image clé contenant le carré.
3. Déplace le carré ailleurs sur la scène, puis clique le bouton ⇄ de l'image clé 1 : interpolation de mouvement.
4. **Espace** : lecture. Le carré glisse.

Tout repose sur trois idées. (1) Un document = des calques d'images clés contenant des éléments (formes, instances de symboles, images, texte). (2) Entre deux images clés avec *tween*, tout s'interpole. (3) Les scripts pilotent le document pendant la lecture.

2. Tour de l'interface

2.1 Barre de menus

Menu	Entrées
Fichier	Nouveau · Ouvrir... (JSON) · Archives · Importer SVG · Importer image... · Enregistrer JSON · Exporter HTML · Compiler un HTML
À propos	Mentions légales & RGPD · Documentation · APK Debug · Code Source

Ouvrir..., Nouveau et Archives ouvrent une nouvelle fenêtre-projet sans rien perdre : chaque projet garde sa timeline, sa bibliothèque, ses scripts et son propre historique annuler/rétablir. La lecture d'un projet se met en pause quand sa fenêtre n'est plus active.

2.2 Barre d'outils

Outil	Raccourci	Usage
Sélection	V	Sélectionner, déplacer, transformer
Sous-sélection	A	Éditer les points d'une courbe
Rectangle / Ellipse	R / O	Formes
Ligne / Plume	L / P	Traits et courbes de Bézier (clic = point anguleux, clic-glissé = point lisse, Entrée = terminer, Échap = annuler)
Pinceau	B	Trait libre lissé
Texte	T	Boîte de texte paragraphe

Suivent les couleurs de remplissage/contour, la taille du pinceau, le zoom et l'outil main (pan). La scène s'adapte à l'écran (mobile/tablette inclus) et passe en plein écran.

2.3 Timeline

- **Calques** : visibilité, verrouillage, renommage, ajout/suppression.
- **Images clés** : **F6** image clé, **F7** image clé vide, **Espace** lecture/pause, scrub sur la règle.
- **Interpolation** : bouton ⇌ sur une image clé — position, rotation, échelle, opacité, couleurs ; 4 courbes d'accélération (Propriétés).

- **Morphing** : si une courbe tweenée a le même nombre de points sur l'image clé suivante, elle se déforme point à point.
- **Étiquette** : champ pour nommer une image (ex. *Start*) — utilisable avec `gotoAndPlay("Start")`.
- **Script d'image** : un calque peut porter du code exécuté quand la tête de lecture atteint cette image (comportement Animate CC : en lecture seulement).

2.4 Bibliothèque et symboles

- Sélectionner des objets puis **F8** : **Convertir en symbole** — type *Graphic* (synchro sur le parent) ou *MovieClip* (timeline indépendante, scripts propres).
- Double-clic sur une instance (ou un élément de la bibliothèque) : édition du symbole en mode isolé.
- Glisser un élément de la bibliothèque sur la scène : nouvelle **instance**. Deux instances du même symbole jouent indépendamment.
- Bouton d'export « objet de jeu » sur un symbole : voir chapitre 8.

2.5 Panneau Propriétés

Contextuel à la sélection : **Nom d'instance**, X/Y, dimensions, rotation, échelle, opacité, remplissage/contour, texte (taille, interligne, alignement). Pour les MovieClips : squelette de skinning. Le **Nom d'instance** est la clé des scripts : c'est lui qui devient une variable.

2.6 Panneau Scripts

Éditeur CodeMirror avec plusieurs scripts nommés par onglets, boutons **Exécuter** (**Ctrl+Entrée**) et Arrêter, et une console de sortie. Le bouton « image » cible le script d'une image précise de la timeline.

3. Les scripts : le contrat

3.1 Qui voit quoi

Un script est compilé et exécuté avec **cinq paramètres** — c'est tout ce qu'il voit :

```
Scene Game console named createjs
```

Paramètre	Contenu
<code>Scene</code> / <code>Game</code>	L'API de pilotage (alias l'un de l'autre) : lecture, dimensions, création d'éléments, événements
<code>console</code>	Console (sortie visible dans le panneau)
<code>named</code>	Map des instances nommées de l'image courante : <code>named["But"]</code>
<code>createjs</code>	La bibliothèque CreateJS complète (EaselJS, TweenJS, SoundJS, PreloadJS) chargée en local

Chaque instance **nommée** est de plus injectée comme **variable directe** : écrire `But.x += 1` fonctionne sans déclaration.

Mode strict. Les scripts s'exécutent en `"use strict"` : `this` vaut `undefined` au niveau supérieur — n'écris pas `this.But`, écris `But`.

3.2 Quand un script s'exécute

- **Scripts nommés** (onglets) : au clic sur Exécuter. Chaque Exécuter repart de zéro (les inscriptions d'événements précédentes sont retirées).
- **Scripts d'image** : automatiquement quand la lecture atteint l'image, à chaque passage.
- Les **événements de souris** ne se déclenchent **qu'en lecture** : en édition, le clic appartient aux outils.

3.3 L'API Scene (essentiel)

```
Scene.width / Scene.height / Scene.frameRate / Scene.frameCount / Scene.backgroundColor  
Scene.play() / Scene.stop()  
Scene.gotoAndPlay(frameOuLabel) / Scene.gotoAndStop(frameOuLabel) // ex. gotoAndStop("Start")  
Scene.currentFrame / Scene.playing / Scene.keys  
Scene.onEnterFrame(cb) / Scene.onKeyDown(cb) / Scene.onKeyUp(cb) / Scene.onClick(...)  
Scene.addShape(type, props) / Scene.addInstance(symbolId, props)  
Scene.log(...) / Scene.random(n)
```

Règle d'or des handlers. Les inscriptions (`But.on("click", ...)`) se font **une seule fois, au niveau supérieur du script** — jamais dans `onEnterFrame`, sous peine d'empilement. Les handlers posés par Exécuter survivent aux scripts d'image ; les handlers posés par un script d'image sont purgés entre ses passages.

3.4 Le quadri-diagnostic (à connaître par cœur)

```
Scene.log("createjs :", typeof createjs);  
Scene.log("nommes :", Object.keys(named).join(", ") || "AUCUN");  
Scene.log("But typeof:", typeof But);  
Scene.log("playing:", Scene.playing);
```

Quand quelque chose ne réagit pas, ces quatre lignes éliminent 90 % des causes : bibliothèque absente, instance invisible à cette image, nom mal orthographié, lecture arrêtée.

4. Exploiter CreateJS dans les scripts

La bibliothèque **CreateJS** complète est disponible sous la variable `createjs` — copie locale (dossier `libs/createjs`, licence MIT), aucune connexion requise.

4.1 TweenJS : les interpolations programmées

```
createjs.Tween.get(cible)           // cible : un élément nommé, ou tout objet
    .to({ x: 300, rotation: 180 }, 1000) // 1 seconde
    .wait(300)                       // pause 300 ms
    .to({ y: 200 }, 500, createjs.Ease.bounceOut)
    .call(function () { Scene.log("fini !"); });
```

Objet	Usage
<code>createjs.Tween</code>	Séquences d'interpolation ; <code>.get(cible)</code> , <code>.to(props, ms, ease)</code> , <code>.wait(ms)</code> , <code>.call(fn)</code> , <code>removeTweens(cible)</code>
<code>createjs.Ease</code>	Courbes : <code>linear</code> , <code>quadOut</code> , <code>cubicInOut</code> , <code>bounceOut</code> , <code>elasticOut</code> , ...
<code>createjs.Ticker</code>	Horloge à ~60 Hz. Mis en pause avec l'éditeur : les tweens s'arrêtent quand tu arrêtes la lecture. <code>e.delta</code> en ms dans le callback.
<code>createjs.Sound</code>	Sons (<code>Sound.registerSound</code> , <code>Sound.play</code>)
<code>createjs.LoadQueue</code>	Chargement de fichiers (images, sons, JSON)

Pourquoi c'est fluide. Le document tourne à son `frameRate` (ex. 24 fps) mais le rendu se fait à chaque frame d'affichage (~60 fps) : un tween CreateJS reste parfaitement lisse, même si la timeline est lente.

4.2 Exemples éclair

```
// Clignotement piloté par le Ticker (60 fps)
createjs.Ticker.addEventListener("tick", function (e) {
    lampe.opacity = 0.5 + 0.5 * Math.sin(e.delta * 0.01);
});

// Chaîne : approche puis rebond
createjs.Tween.get(Balle)
    .to({ x: 400 }, 800, createjs.Ease.quadIn)
    .to({ x: 0 }, 1200, createjs.Ease.bounceOut);
```

5. Événements et interactions

5.1 Sur une instance nommée (sémantique EaselJS)

```
But.on("click", function (ev) {  
    Scene.log("clic en", ev.stageX, ev.stageY, "sur", ev.target.name);  
});
```

Événement	Émis quand
<code>click</code>	Down et up sur le même élément (règle CreateJS)
<code>mousedown</code> / <code>mouseup</code> / <code>dblclick</code>	Sur l'élément sous le curseur
<code>pressmove</code> / <code>pressup</code>	Sur l'élément pressé — pendant et après la pression (base du glisser-déposer)
<code>mouseover/out</code> , <code>rollover/rollout</code>	Entrée/sortie du survol
<code>tick</code>	À chaque image de lecture

Également : `off` / `removeEventListener`, `removeAllEventListeners`, `hasEventListener`, `dispatchEvent`.
Sur `Scene` : `stagemousedown/stagemousemove/stagemouseup/stageclick` (tout clic, même dans le vide) et `tick`.

5.2 Scene.onClick — la version rapide

```
Scene.onClick(function (x, y) { ... }); // tout clic sur la scène  
Scene.onClick("But", function (x, y, el) { ... }); // clic sur l'instance nommée
```

5.3 Où peut-on cliquer ? (hit-testing)

- **Ellipse** : le disque réel, pas la boîte englobante.
- **Rect, texte, image** : boîte englobante ; rotation et échelle prises en compte.
- **Ligne/chemin** : boîte des points.
- **Instance de symbole** : enveloppe de tout ce que dessine le symbole (toutes ses images). Un symbole vide n'est cliquable nulle part.
- Les éléments d'un calque tweené sont testés à la position de leur image clé (approximation).

5.4 Navigation par labels

```
Scene.gotoAndPlay("Menu");  
But.on("click", function () { Scene.gotoAndStop("Start"); });
```

Les labels se posent dans la timeline (champ Étiquette). Un label introuvable produit un avertissement console et aucun déplacement. Dans un script posé à *l'intérieur* d'un MovieClip, les labels résolus sont ceux du symbole.

6. Recettes prêtes à tester

Pour chaque recette : nomme une instance comme indiqué (champ « Nom d'instance » des Propriétés), colle le code dans le panneau Scripts, **Exécute**, et mets la lecture.

Recette 1 — Diagnostic universel

```
Scene.log("createjs :", typeof createjs);
Scene.log("nommes :", Object.keys(named).join(", ") || "AUCUN");
Scene.play();
```

Résultat : la console affiche l'état du contrat. Aucune condition.

Recette 2 — Bouton de section (le patron « menu »)

```
Scene.play();
But.on("click", function () {
    Scene.gotoAndStop("Start");
});
```

Résultat : au clic sur l'instance `But` (pendant la lecture), la tête de lecture saute à l'image étiquetée *Start* et s'y arrête. Variante visuelle : change `But.scaleX/Y` à `mousedown` et remets à 1 à `pressup`.

Recette 3 — Glisser-déposer borné

```
Scene.play();
var off = null;
But.on("mousedown", function (ev) { off = { dx: But.x - ev.stageX, dy: But.y - ev.stageY }; });
But.on("pressmove", function (ev) {
    if (!off) return;
    But.x = Math.max(0, Math.min(Scene.width, ev.stageX + off.dx));
    But.y = Math.max(0, Math.min(Scene.height, ev.stageY + off.dy));
});
But.on("pressup", function () { off = null; });
```

Résultat : l'instance suit le doigt/souris sans sauter au centre, et ne sort jamais de la scène.

Recette 4 — Déplacement au clavier

```
Scene.play();
Scene.onEnterFrame(function () {
    var v = 6;
    if (Scene.keys["ArrowLeft"]) Perso.x -= v;
    if (Scene.keys["ArrowRight"]) Perso.x += v;
    if (Scene.keys["ArrowUp"]) Perso.y -= v;
    if (Scene.keys["ArrowDown"]) Perso.y += v;
});
```

Résultat : l'instance `Perso` bouge aux flèches pendant la lecture. (La saisie dans les panneaux est ignorée automatiquement.)

Recette 5 — Tween CreateJS avec suite logique

```
Scene.play();
createjs.Tween.get(Balle)
    .to({ x: 400 }, 800, createjs.Ease.quadIn)
    .to({ x: 0 }, 1200, createjs.Ease.bounceOut)
    .call(function () { Scene.log("Terminé, retour au départ"); });
```

Résultat : la balle accélère vers la droite puis rebondit au retour, et la console confirme la fin.

Recette 6 — Compteur de score avec tick

```
Scene.play();
var score = 0;
Score.on("tick", function () {
    score = score + 1;
    Score.text = "Score : " + score;
});
```

Résultat : une instance de **texte** nommée `Score` s'incrémente à chaque image de lecture. (Remarque : le handler est posé par Exécuter, il survit aux scripts d'image.)

Recette 7 — Élément qui suit la souris

```
Scene.play();
Scene.on("stagemousemove", function (ev) {
    Viseur.x = ev.stageX;
    Viseur.y = ev.stageY;
});
```

Résultat : l'instance `Viseur` se colle au curseur pendant la lecture.

Recette 8 — Menu complet (trois boutons)

```
Scene.play();
B1.on("click", function () { Scene.gotoAndStop("Jeu"); });
B2.on("click", function () { Scene.gotoAndStop("Score"); });
B3.on("click", function () { Scene.gotoAndStop("Options"); });
```

Résultat : chaque bouton saute à sa section. Un bouton « Retour » posé sur chaque section peut être la même instance nommée `Retour` pointant vers `"Menu"`.

7. Exporter : JSON, HTML, compilé

Format	Contenu	Quand l'utiliser
Enregistrer JSON / Ouvrir...	Le projet complet	Sauvegarde, reprise de travail
Exporter HTML	Un fichier HTML autonome qui rejoue l'animation, avec les scripts (exécutés dynamiquement) ; CreateJS inliné si un script l'utilise	Partager une animation jouable, l'intégrer quelque part
Compiler un HTML	Un fichier HTML où les scripts sont de vraies fonctions dans de vraies balises — zéro eval, preuve embarquée par une CSP qui l'interdit ; données en bloc JSON pur	Livrer du code lisible et verrouillé ; environnements stricts
Objet de jeu (bouton sur un symbole)	<code>tween-runtime.js</code> + une classe JS par symbole	Chapitre 8 — intégrer dans votre propre jeu

Parité. Les deux exports HTML se comportent comme l'éditeur : mêmes événements (clics en lecture), mêmes labels, même `Scene.onClick`. L'export classique exécute vos scripts dynamiquement (comme l'éditeur) ; le compilé les a transformés en fonctions réelles — ouvrez-le dans un éditeur de texte, votre code y est lisible.

8. Utiliser un MovieClip dans votre propre jeu

C'est la sortie pensée pour les développeurs : animez un personnage dans Animate JS (labels « walk », « jump »...), exportez-le, et pilotez-le depuis **votre** code JavaScript, dans n'importe quelle page ou moteur de jeu.

8.1 Exporter

1. Cliquez le bouton « **objet de jeu** » sur le symbole dans la bibliothèque.
2. Deux fichiers sont téléchargés : `tween-runtime.js` (le moteur, aucune dépendance) et `MonPerso.js` — une classe qui embarque les données du symbole et de tout ce dont il dépend.

```
// MonPerso.js – généré par Animate JS
import { MovieClip } from './tween-runtime.js';

const DATA = { /* ...calques, images clés, labels, bitmaps... */ };

export class MonPersoClip extends MovieClip {
  constructor(props) { super(DATA, props); }
}

// Labels disponibles pour gotoAndPlay()/gotoAndStop() : "walk", "jump"
```

8.2 L'API MovieClip

Membre	Rôle
<code>x, y, rotation, scaleX, scaleY, opacity, visible, loop</code>	Transformations — modifiables à tout moment
<code>play() / stop()</code>	Lecture de la timeline du clip
<code>gotoAndPlay(frameOuLabel) / gotoAndStop(frameOuLabel)</code>	Saut direct (numéro ou label)
<code>currentFrame / frameCount / isPlaying</code>	État (lecture seule)
<code>update(dt)</code>	Avance l'animation — <code>dt</code> en millisecondes ; à appeler une fois par frame de jeu
<code>draw(ctx)</code>	Dessine sur un contexte Canvas 2D — après <code>update</code>
<code>addEventListener("loop" / "complete", fn)</code>	Boucle terminée / fin de lecture sans boucle
imbriqués	Les MovieClips enfants ont leur propre timeline et avancent avec le parent

8.3 Exemple complet — page de jeu autonome

```
<!doctype html>
<html lang="fr">
<body>
<canvas id="jeu" width="640" height="480"></canvas>
<script type="module">
  import { MonPersoClip } from './MonPerso.js';

  const canvas = document.getElementById('jeu');
  const ctx = canvas.getContext('2d');

  const perso = new MonPersoClip({ x: 100, y: 300 });
  perso.gotoAndPlay('walk'); // label posé dans l'éditeur

  perso.addEventListener('complete', function () {
    console.log('Animation terminée (loop = false)');
  });

  addEventListener('keydown', function (e) {
    if (e.key === ' ') perso.gotoAndPlay('jump'); // Espace : saut
  });

  let last = null;
  function gameLoop(t) {
    requestAnimationFrame(gameLoop);
    if (last === null) last = t;
    const dt = t - last; // ms
    last = t;

    perso.update(dt); // 1. faire avancer l'animation
    perso.x += 1; // 2. votre logique de jeu
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    perso.draw(ctx); // 3. dessiner
  }
  requestAnimationFrame(gameLoop);
</script>
</body>
</html>
```

Les trois règles d'intégration. (1) `dt` est en **millisecondes**. (2) Un seul `update(dt)` par frame, `draw(ctx)` juste après. (3) Chaque instance a son état propre : créez deux `new MonPersoClip()` pour deux ennemis indépendants.

9. Dépannage

Symptôme	Cause probable	Remède
<code>ReferenceError: X is not defined</code>	L'instance n'existe pas à l'image courante, ou le nom diffère (casse, espace)	Quadri-diagnostic ; vérifier le Nom d'instance et l'image
Le clic ne fait rien	Lecture arrêtée, handler inscrit dans <code>onEnterFrame</code> , ou clic hors zone (hit-test)	Lecture ; handler au top-level ; vérifier le contour du symbole
Le clic déclenche N fois	Inscription répétée à chaque image	Déplacer l'inscription au niveau supérieur du script
Handler muet malgré tout	Un script d'image sur un autre calque (ancien bug, corrigé)	Mettre l'application à jour ; sinon quadri-diagnostic + console
<code>createjs</code> est <code>undefined</code>	Vieille version déployée	Reconstruire / redéployer les deux versions
L'export HTML est blanc	Vieille version (bug corrigé)	Re-exporter depuis une version à jour

10. Annexes

10.1 Raccourcis clavier

Touche	Action
V A R O L P B T	Outils (sélection, sous-sélection, rectangle, ellipse, ligne, plume, pinceau, texte)
F6 / F7	Image clé / image clé vide
F8	Convertir en symbole
Espace	Lecture / pause
Ctrl+Entrée	Exécuter le script
Échap	Annuler le tracé en cours (plume, pinceau)

10.2 Limites connues (octobre 2026)

- Hit-test des éléments **tweenés** : position de l'image clé (approximation).
- Pinceau : un seul pattern (trait lisse), pas de sensibilité à la pression.
- Les événements de scripts ne vivent qu'en lecture (choix assumé, comme les scripts d'image d'Animate CC).
- L'« objet de jeu » exporte un symbole et ses dépendances, pas toute la scène racine.

10.3 Pour aller plus loin

- **Cours CreateJS v1 et v2** (dossier [docs/](#)) : l'intégration expliquée de l'intérieur, avec étude de cas complète.
- **Documentation scripts** ([Scripts-Programmation.pdf](#)) : référence détaillée de l'API Scene.
- **CreateJS** : documentation officielle des quatre bibliothèques (MIT).